رواد مصر الرقمية

وزارة الاتصالات
وتكنولوجيا المعلومات

Data Preparation for Exploration

Session 2

Contents

Contents

- DML (Data Manipulation Language)
  - SELECT Command
  - INSERT Command
  - UPDATE Command
  - DELETE Command
- DISTINCT Keyword
- Comparison Conditions
- Logical & Arithmetic Expressions
- ORDER BY Clause

رواد مصر الرقمية

وزارة الاتصالات وتكنولوجيا المعلومات

Contents

- Joins
  - Inner Join
  - Outer Join
  - Right Join
  - Left Join
  - Self Join
  - Cartesian Product
- UNIONS
- CASE WHEN
- Sub-Queries
- Aggregate Functions & GROUP BY Clause

Contents

- CTE (Common Table Expression)
- Views

Agenda

- Database Definition
- DBMS (Database Management Systems)
- Database Transaction
- Database Schema

File Based System

It is a collection of programs that perform services for the end user.

Each program defines and manages its own data.

Limitations of File-Based Systems:

Separation & Isolation of Data:

Data is stored in multiple files, leading to silos.

Difficulty in accessing related data across different files.

Duplication of Data:

Redundant data entries across files increase storage costs and the potential for inconsistencies.

Challenges in maintaining data integrity.

Program Data Dependence:

Applications are tightly coupled with data structures, making it difficult to modify data formats without affecting the program.

Requires constant updates to applications with data structure changes.

Incompatible File Formats:

Various applications may use different file formats, hindering data sharing and collaboration.

Increases the complexity of data integration efforts.

What is a Database?

- **What is a Database?**
  A **database** is a **collection of related data** organized in a structured way to enable efficient storage, retrieval, and management. Databases are designed to handle various types of data and support operations such as querying, updating, and reporting.

What is a Database?

- **Examples of Databases**:
- **To-Do List**: Stores tasks that need to be completed, often with attributes like task names, due dates, and priority levels.
- **Shopping List**: Keeps track of items to be purchased, including item names, quantities, and categories.
- **Your Best Books**: Maintains information about books, such as titles, authors, genres, and personal ratings.
- **Company's Employee Base**: Manages employee data, including names, job titles, departments, and contact details.

What is a Database?

Database is a collection of related data

What is a Database? (Cont.)

- **On Paper**: Data is recorded on physical documents or forms. This method is less common for complex data management due to its limitations in storage capacity and retrieval efficiency.

- **In Your Mind**: Informal data storage where information is kept mentally. This approach is not practical for handling large or complex datasets and is prone to memory limitations and inaccuracies.

- **On a Computer**: Data is stored electronically using various database management systems (DBMS). This method allows for efficient data storage, retrieval, management, and analysis, and supports advanced functionalities such as querying and reporting.

Working with paper

Working with paper

- Lack of storage space
- Difficulty searching for information
- Security issues
- Document transportation
- High costs

Working with Paper

- **Lack of Storage Space**: Physical documents require significant space for storage, which can become impractical as the volume of data grows.
- **Difficulty Searching for Information**: Finding specific information in paper records can be time-consuming and inefficient compared to digital search capabilities.
- **Security Issues**: Paper records are vulnerable to loss, theft, and unauthorized access, posing risks to data security and confidentiality.
- **Document Transportation**: Moving physical documents between locations can be cumbersome and may lead to damage or loss.
- **High Costs**: The costs of printing, storing, and managing paper documents can add up, making it a more expensive option compared to digital storage.

Database Management Systems (DBMS)

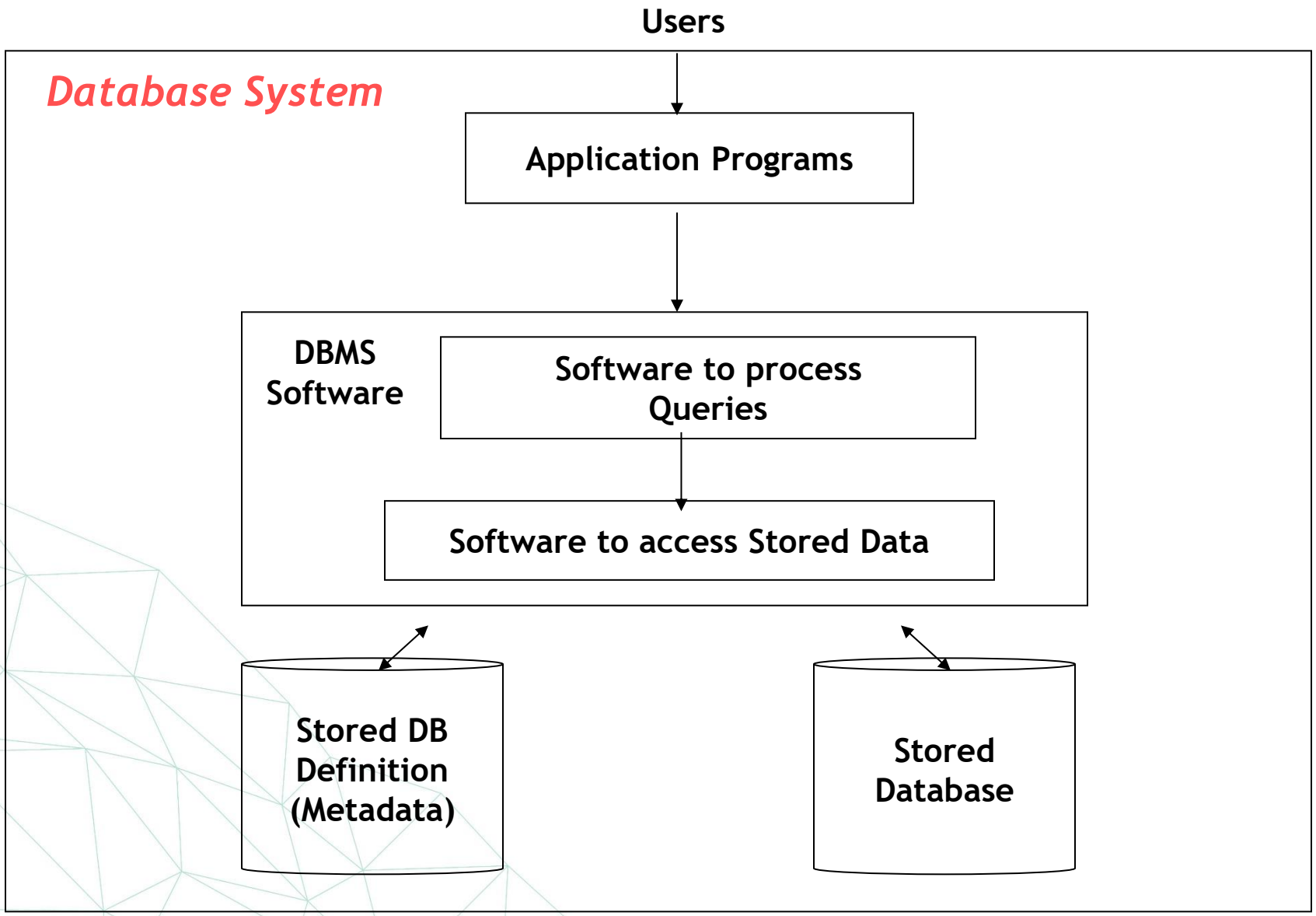
A **Database Management System (DBMS)** is a collection of programs that **manages the database structure** and **controls access** to the data stored in the database.

Advantages of DBMS:

- **Easier Management of Large Amounts of Data**: DBMS provides tools and features to handle and organize large datasets efficiently.
- **Better Data Security**: Implements access controls and encryption to protect data from unauthorized access and breaches.
- **Minimized Data Inconsistency**: Ensures data accuracy and consistency through centralized management and validation rules.
- **Reduction in Data Redundancy**: Eliminates duplicate data by using normalization techniques, reducing storage requirements and improving data integrity.
- **Faster Data Access**: Utilizes indexing and optimized querying techniques to enhance data retrieval speed.
- **Interaction with Software Applications**: Supports interaction with various programming languages and software applications, facilitating integration and data manipulation.

Database System

- **Database System**: The DBMS software together with the data itself. Sometimes, the applications are also included. (Software + Database)


Users
Database System
Application Programs
DBMS Software
Software to process Queries
Software to access Stored Data
Stored DB Definition (Metadata)
Stored Database

DBMS Advantages

- Controlling Redundancy.
- Restricting Unauthorized Access.
- Sharing data.
- Enforcing Integrity Constraints
- Inconsistency can be avoided.
- Providing Backup and Recovery.

DBMS Other Functions

Data mining

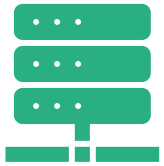
Spatial Data

Image / Audio / Video

Time Series

Database Environment

Mainframe environment



Client/Server environment

Internet Computing environment

Data Models

- Provide concepts that are close to the way many users perceive data, entities, attributes, and relationships. (Ex. ERD)
- Describes how data is stored in the computer and the access path needed to access and search for data.

Database Tables

Students

Primary Key

ID	Student_name	Age
1	Ahmed	24
2	Mohamed	22
3	Amira	25
4	Ahmed	24

Database Tables

transactions

transaction_id	product_id	spend
131432	8476	1474.00
131433	1324	501.00
247826	3677	1001.20

Database Tables

Employee

Emp_id	First_name	Last_name	Gender	Salary
100	Ibrahim	Ahmed	M	7000
101	Naglaa	Mohamed	F	9000
102	Khaled	Saied	M	8300

Database Tables

Foreign Key

Employee

Emp_id	First_name	Last_name	Gender	Salary	Dept_id
100	Ibrahim	Ahmed	M	7000	3
101	Naglaa	Mohamed	F	9000	1
102	Khaled	Saied	M	8300	2

Departments

ID	Name
1	HR
2	Data
3	IT

Database Tables

Employee

Emp_id	First_name	Last_name	Gender	Salary	SV_id
100	Ibrahim	Ahmed	M	7000	101
101	Naglaa	Mohamed	F	10000	104
102	Khaled	Saied	M	8300	101
103	Ahmed	Essa	M	6500	101
104	Reem	Ragab	M	14000	N/A

Database Types

Relational Databases
Organize data into **tables** with **rows** and **columns**, where relationships between different tables are defined using **keys**.
Use Cases: **Transaction processing**, **business applications**, and **data warehousing**.

Non-Relational (NoSQL) Databases
Use flexible data models, such as **document**, **key-value**, **wide-column**, or **graph** structures, instead of traditional tables.

Distributed Databases
Distribute data across **multiple nodes** in a network, allowing for **horizontal scaling** and **fault tolerance**.

Database Types

1. **Relational Databases:**

   Organize data into tables (rows and columns) with relationships defined between different tables using keys.

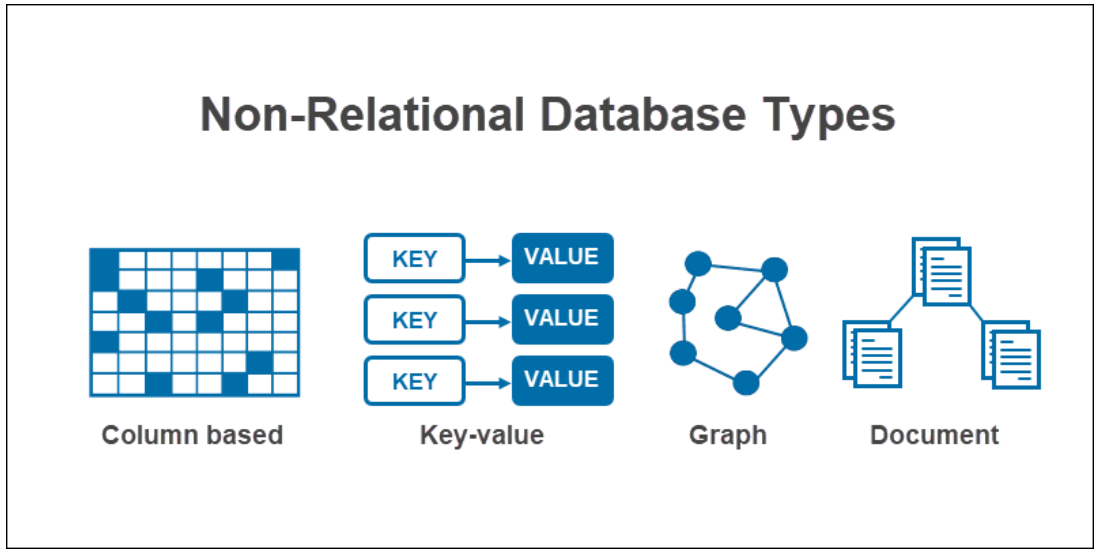
   - **Use Cases:** Transaction processing, business applications, and data warehousing.

Database Types

2. Non-Relational (NoSQL) Databases:

Use flexible data models, such as document, key-value, wide-column, or graph structures, instead of tables.

- **Use Cases:** Big data analytics, real-time applications, content management systems.

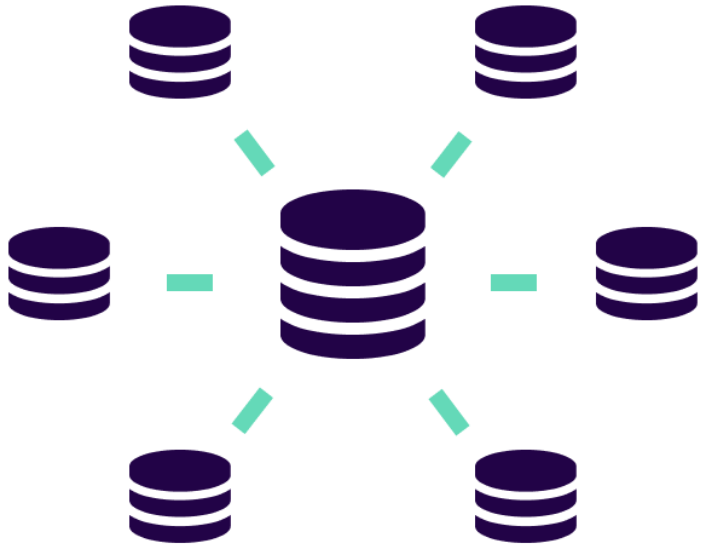


Database Types

3. Distributed Databases:

Distribute data across multiple nodes in a network, allowing horizontal scaling and fault tolerance.

- **Use Cases:** Cloud-based services, large-scale applications requiring high availability.



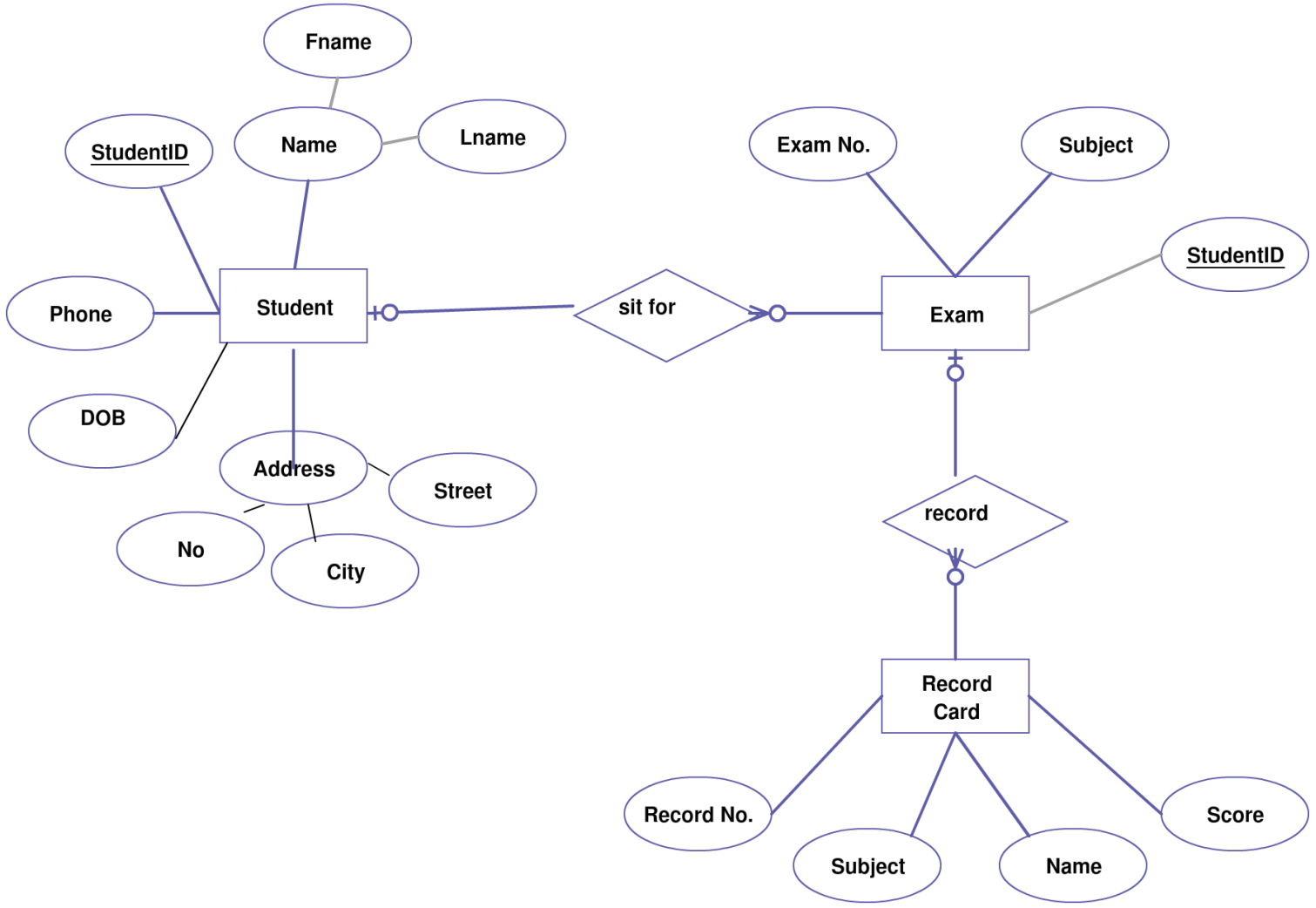
MySQL vs. MongoDB

Feature	MongoDB (Non-Relational)	MySQL (Relational)
Data Structure	JSON-like documents	Tables with rows and columns
Schema	Flexible schema (no fixed structure)	Fixed schema (requires predefined structure)
Query Language	JSON-like syntax for queries	SQL (Structured Query Language)
ACID Compliance	Supports eventual consistency; limited ACID in some cases	Fully supports ACID transactions
Scalability	Horizontally scalable (scale-out)	Vertically scalable (scale-up)
Performance	Optimized for large volumes and fast performance	Good for complex queries; may slow down with large datasets
Ideal Use Cases	Real-time analytics, content management, IoT	Banking, eCommerce, CRM

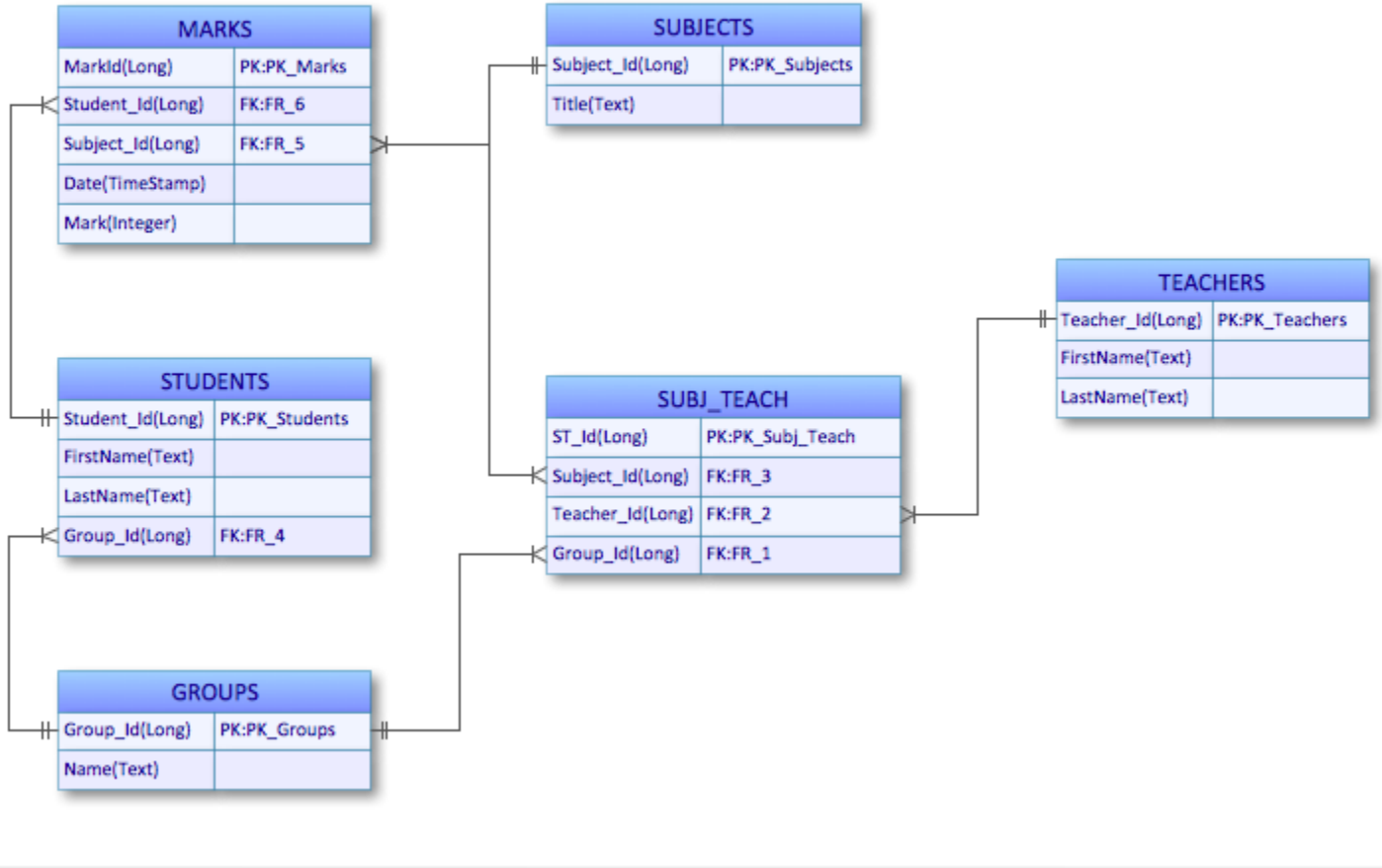
Entity Relationship Modeling

Entity-Relationship Diagram (ERD):

identifies information required by the business by displaying the relevant entities and the relationships between them.


Fname
StudentID
Name
Lname
Phone
Student
sit for
Exam No.
Subject
StudentID
Exam
DOB
Address
Street
No
City
record
Record Card
Record No.
Subject
Name
Score

Entity-Relationship Diagram using Crow's Foot notation



Definitions

An **entity** is a thing in the real world with an **independent existence**. It can have a **physical existence** (e.g., a particular person or car) or a **conceptual existence** (e.g., a job or a university course).

An **attribute** refers to the particular **properties** that describe an entity. For example, an **EMPLOYEE** entity may be described by attributes such as **name**, **age**, **address**, and **salary**.

An **entity instance** is a specific **occurrence** of an entity. For example, each individual person or car represents an instance of the **Person** or **Car** entity, respectively.

Types of Attributes

A **key** is an attribute whose values are **distinct (unique)** for each entity and can be used to **uniquely identify** the record. For example, an **Employee ID** can be a key in an employee database.

A **multi-valued attribute** has a set of **values** for the same entity instance. For example, an **Employee** might have multiple **phone numbers**.

A **derived attribute** can be **calculated** from another attribute or entity. For example, an **Employee's age** can be derived from the **date of birth** attribute.

A **single/simple attribute** is **not divisible** and has a single value for a particular entity instance. For example, an **Employee's name** is a single attribute.

Relationships

- **Relationships** - A relationship is a connection between entity classes.
- Establishes a connection or correspondence or link between a pair of tables in a database, or between a pair of entities in an entity-relationship diagram (ERD).

Relationships (cont.)

- A **one-to-one relationship** occurs when a **single record** in **Table A** is related to only **one record** in **Table B**, and vice versa. For example, each employee may have only one employee ID, and each employee ID corresponds to exactly one employee.

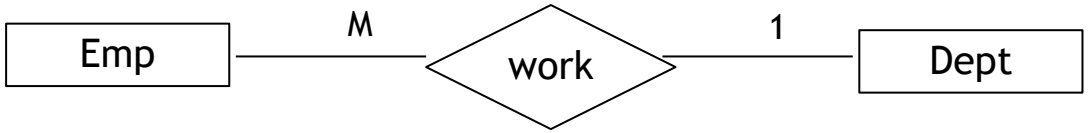
- **One-to-Many Relationship**
  A **one-to-many relationship** occurs when a **single record** in **Table A** can be related to **one or more records** in **Table B**, but a **single record** in **Table B** is related to only **one record** in **Table A**. For example, a single customer (Table A) can have multiple orders (Table B), but each order is linked to only one customer.

- **Many-to-Many Relationship**
  A **many-to-many relationship** occurs when a **single record** in **Table A** can be related to **one or more records** in **Table B**, and vice versa.

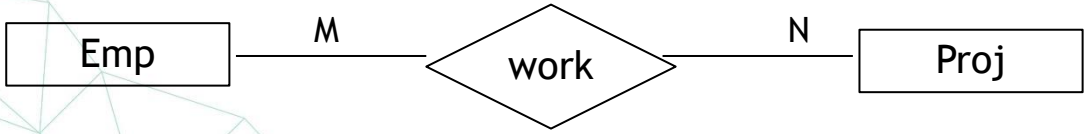
Relationships (cont.)

a. One to one

b. One to many



c. Many to many



Key Types

- **Primary Key**
  A **primary key** is used to ensure that data in a specific column is **unique** and that **NULL values** are not allowed. It is either an **existing table column** or a column that is **specifically generated** by the database according to a defined sequence.

- **Foreign Key**
  A **foreign key** is a column in a relational database table that provides a **link** between data in two tables. It is a column that **references** a column (most often the primary key) of another table.

Foreign Key

Employee

Emp_id	First_name	Last_name	Gender	Salary	Dept_id
100	Ibrahim	Ahmed	M	7000	3
101	Naglaa	Mohamed	F	9000	1
102	Khaled	Saied	M	8300	2

Departments

ID	Name
1	HR
2	Data
3	IT

ERD Notations

- **Rectangles** represent ENTITY CLASSES
- **Circles** represent ATTRIBUTES
- **Diamonds** represent RELATIONSHIPS
- **Arcs** - Arcs connect entities to relationships.
- **Underline** - Key attributes of entities are underlined.

Entity Relationship Modeling

Entity-Relationship Diagram (ERD):

identifies information required by the business by displaying the relevant entities and the relationships between them.

Database Schema

A schema is a group of related objects in a database.

There is one owner of a schema who has access to manipulate the structure of any object in the schema. A schema does not represent a person, although the schema is associated with a user that resides in the database.

What is SQL?

C R U D

Create Read Update Delete

What is SQL?

- **SQL** is a language used for **interacting with DBMS**. You can use it to:
- **Create**, **Read**, **Update**, and **Delete** data
- **Create** and **Manage** databases
- **Design** and **Create** tables

- There are different **DBMS** and each has a **different implementation**:
- **MySQL**, **PostgreSQL**, **T-SQL**, etc.
- **Same Concepts** but **different implementation**

Structured Query Language (SQL)

- Data Definition Language (DDL)
- Data Manipulation Language (DML)
- Data Control Language (DCL)

How to Apply Data Validation?

1. Data Types
2. Database Constraints

Data Types

A data type determines the type of data that can be stored in a database column. The most used data types are:

- Alphanumeric: data types used to store characters, numbers, special characters, or nearly any combination.
- Numeric
- Date and Time

SQL Data Types

- **Alphanumeric:** This type of data stores character values.
- **CHAR(n)** - used for data(string) with a fixed-length of characters.
- **VARCHAR(n)** - the variable-length character string.
  Similar to CHAR(n), it can store “n” length data
- **TEXT** - the variable-length character string. It can store data with unlimited length.

SQL Data Types

- **Numeric**: Used to store numerical values. Examples include:
- **INTEGER**: Whole numbers.
- **DECIMAL/NUMERIC**: Numbers with decimal points.
- **FLOAT/DOUBLE**: Floating-point numbers for precision.

SQL Data Types

- **Date and Time**: Used to store date and time values. Examples include:

- **DATE**: Dates (e.g., YYYY-MM-DD).
- **TIME**: Time of day (e.g., HH:MM).
- **DATETIME/TIMESTAMP**: Combined date and time values.

Database Constraints

- Primary Key (Not Null + Unique)
- Not Null
- Unique Key
- Referential Integrity (FK)
- Check

Database Constraints (Cont.)

NOT NULL: Ensures that values in a column **cannot be NULL**. Every record must have a value for this column.

UNIQUE: Ensures that values in a column are **unique across all rows** within the same table. No two rows can have the same value for this column.

PRIMARY KEY: A **primary key column** uniquely **identifies rows** in a table. A table can have only one primary key, which may consist of one or more columns.

CHECK: A **CHECK constraint** ensures that the data in a column **must satisfy a Boolean expression**. It enforces specific rules or conditions on column values.

FOREIGN KEY: Ensures that values in a column or group of columns in one table must **exist** in a column or group of columns in another table. This enforces **referential integrity** between tables. A table can have multiple foreign keys.

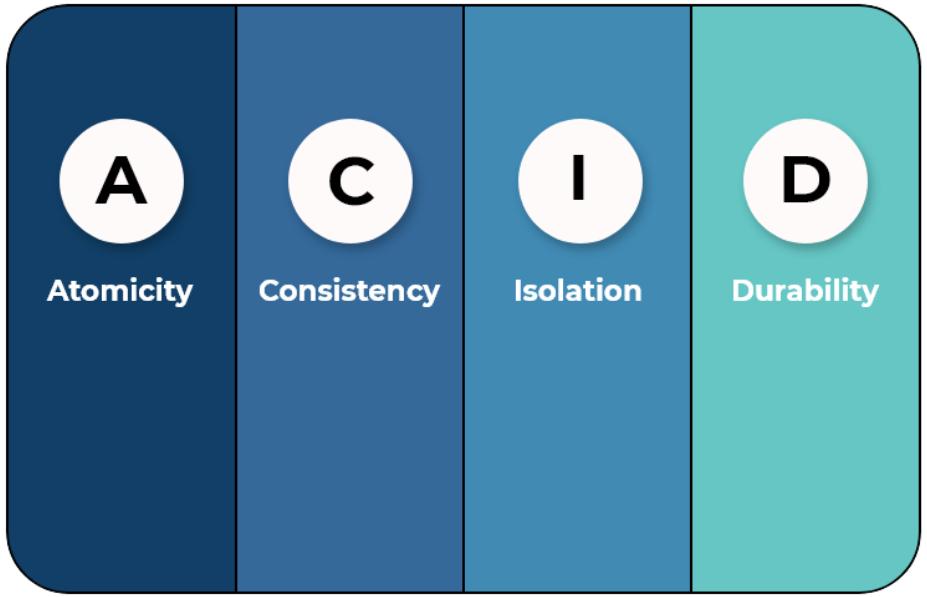
Database Transaction

- A transaction is an executing program that forms a logical unit of database actions.
- It includes one or more database access operations such as insert, delete and update.
- The database operations that form a transaction can either be embedded within an application program or they can be specified interactively via a high-level query language such as SQL.

Database Transaction Properties

- Transactions should possess several properties, often called the ACID properties:

1. Atomicity
2. Consistency
3. Isolation
4. Durability



Data Organization and Cleaning Best Practices

Efficient data organization and cleaning are crucial for ensuring the quality, integrity, and reliability of data used in analysis and decision-making processes. Properly structured and clean data enables faster analysis, accurate insights, and better decision-making.

Data Organization and Cleaning Best Practices

1- Data Structuring: Organizing data into tables, matrices, or hierarchical structures based on the nature of the data and analysis requirements.

Example: Organizing customer data into tables with columns for personal information, purchase history, and preferences.

Best Practices:

- Use consistent data formats.
- Apply meaningful naming conventions.
- Align data structuring with analysis needs to facilitate easy retrieval and processing.

Data Organization and Cleaning Best Practices

2- Indexing: Creating indexes on database tables to optimize data retrieval performance.

Example: Indexing a "Customer_ID" column to speed up search queries.

How It Works: Indexes provide a quick lookup method to retrieve data, much like the index of a book**.**

Techniques: B-trees, hash indexes, and bitmap indexes are common indexing techniques that improve query performance.

Data Organization and Cleaning Best Practices

3- Handling Missing Values: Organizing data into tables, matrices, or hierarchical structures based on the nature of the data and analysis requirements.

Imputation: Replace missing values with estimated values based on statistical techniques or domain knowledge**.**

Example: Filling in missing age data in a customer dataset with the median age.

Importance: Ensures completeness and prevents bias in the analysis.

Data Organization and Cleaning Best Practices

4- Handling Outliers

Visual Inspection: Identify outliers using scatter plots or box plots.

Statistical Tests: Use tests such as Z-scores to detect outliers.

Trimming/Winsorizing: Modify outliers to reduce their impact or remove them if justified.

Example: In a dataset of monthly sales, an unusually high value might be an outlier that needs investigation or adjustment.

Data Organization and Cleaning Best Practices

5- Removing Duplicate Data

Deduplication Algorithms: Identify and merge duplicate records**.**

Fuzzy Matching: Match records with slight differences (e.g., spelling errors) to identify duplicates.

Record Linkage: Combine information from different sources to identify and remove duplicates.

Example: In a mailing list, removing duplicate email addresses ensures each customer receives only one email.

Any Questions ?

Thank You